Foundations of Blockchains

Smart-Contract Lab Instructions

Due: _____

Name: _____ **Section:** _____

Overview: Choose One Question

Complete **one** of the three lab questions below. The purpose of this lab is not merely to make a contract compile: you must deploy it to Remix's local test blockchain, interact with it, and provide visible evidence that its main security behavior works. Read the selected lab's `README.md` and the comments around every `TODO` or multiple-choice decision before editing the Solidity files.

Common Remix workflow

1. Open remix.ethereum.org in Chrome or Firefox. In File Explorer, use **Clone** to load the course repository, or create a workspace and copy in the files named under your selected question.
2. Open the **Solidity Compiler** tab. Select compiler version **0.8.20 or newer** and compile the main contract together with its imports. Resolve every red compiler error before continuing.
3. Open **Deploy & Run Transactions**. Set **Environment** to **Remix VM**; do not use a public network or real funds. Select the named contract, provide any constructor arguments, and click **Deploy**.
4. Expand the instance under **Deployed Contracts** and perform the functional check specified for your question. If you change the source, compile again and deploy a *new* instance: an old deployment still contains the old bytecode.

Screenshot submission. Submit clear, labeled screenshots showing (1) a successful compile with the contract name and compiler version visible, (2) the contract under **Deployed Contracts** with its address visible, and (3) the successful functional check described below. One screenshot may satisfy more than one item if all required evidence is readable. Keep the Remix transaction terminal visible when it helps establish that a call succeeded; a source-code screenshot by itself is not proof of deployment. Paste the final images into the framed placeholders on your selected question's submission page. You may leave the other two question pages blank or omit them from your submitted copy. Confirm that every screenshot is from your final compiled version and does not present a failed transaction as a successful result.

Question 1: NFT Marketplace Auction

Background. This question builds an on-chain auction from three pieces: an ERC-20 payment token, an ERC-721 NFT collection, and a marketplace contract. The marketplace escrows the seller's NFT, pulls payment tokens from bidders using allowances, refunds an outbid participant, and settles after the deadline. The final checkpoint changes the payment rule from a first-price auction to a second-price auction, where the winner pays the second-highest bid and receives the excess escrow back.

Work to complete. Follow `lab/opt3/README.md`.

Complete `Q1--Q20` in `ERC20.sol`, `NFTCollection.sol`, and `Marketplace.sol`, then complete the three code blanks in `SecondPriceMarketplace.sol`. In Remix, open the four files under `lab/opt3/src/` and compile them with version 0.8.20 or newer. Allow Remix to fetch the OpenZeppelin ERC-721 dependency.

Deploy and prepare an auction. Using the first Remix VM account as the seller, deploy `ERC20` with an initial supply, name, and symbol; deploy `NFTCollection`; and deploy `Marketplace` with a market name. In the NFT contract, call `mintNFT` and note the new token ID. Approve the marketplace address to move that NFT. Then call `createAuction` with the NFT address, payment-token address, token ID, a positive starting price, and a Unix end time at least several minutes in the future.

Functional check. Transfer payment tokens to a second Remix VM account. Switch to that account, approve the marketplace to spend more than the intended bid, and call `bid` with auction index 0 and a price strictly above the starting price. Call `getCurrentBidOwner(0)` and `getCurrentBid(0)`. The returned owner must be the bidder and the price must equal the new bid. Also verify with `ownerOf(tokenId)` that the marketplace, rather than the seller, holds the NFT while the auction is open. Your screenshots must show the three deployed contract addresses, the successful `createAuction` and `bid` transactions, and the three read results proving that the NFT and bid are held in escrow as intended.

Question 1 Screenshot Submission

Paste one readable image into each frame. Keep addresses and function results large enough to be checked without zooming into the source code.

Figure 1A: Compile and deployments

Paste screenshot here

Show a successful compile and the deployed addresses of `ERC20`, `NFTCollection`, and `Marketplace`.

Figure 1B: Auction creation and NFT escrow

Paste screenshot here

Show the successful `createAuction` transaction and the `ownerOf(tokenId)` result equal to the marketplace address.

Figure 1C: Bid and marketplace state

Paste screenshot here

Show the successful `bid` transaction together with `getCurrentBidOwner(0)` and `getCurrentBid(0)` returning the expected bidder and price.

Question 2: Hash-Time-Locked ETH and NFT Exchange

Background. A hash-time-locked contract (HTLC) locks an asset with a secret and a deadline. The receiver can claim the asset only by revealing a secret whose hash matches the stored hash. If the secret is not revealed in time, the original sender can take the asset back. The ETH contract and NFT contract are on two different chains. They use the same hash, so revealing the secret to claim one asset makes the secret available for claiming the other. The deadlines must still leave enough time for the second claim.

Work to complete. All Question 2 files are in `lab/opt2/remix/`. First complete `TODO 1--7` in `SimpleHTLC.sol` and compile it with `SimpleHTLCChecker.sol`. Next complete `Q1--Q8` in `SimpleNFTHTLC.sol`, then compile all five Solidity files in the same folder. Complete the ETH checker before running the NFT checker because the final scenario imports your completed ETH HTLC as the ETH contract in the atomic sale.

Functional check, ETH contract. In Remix VM, deploy `SimpleHTLCChecker`. Enter **3** in the **VALUE** field and select **wei**, then call `runAllTests`. Call `isSolved`; it must return `true`. A revert names the behavior that remains incorrect. After a fix, recompile and deploy a fresh checker.

Functional check, NFT contract and atomic sale. Deploy `SimpleNFTHTLCChecker`. Enter **1** in the **VALUE** field with unit **wei**, call `runAllTests`, and then call `isSolved`; it must return `true`. This checker verifies NFT locking, rejection of an incorrect secret, receiver withdrawal, sender refund after expiry, and the linked ETH-for-NFT sale. Your screenshots must show both checkers and both `isSolved = true` results, with the successful `runAllTests` transactions visible.

Question 2 Screenshot Submission

Paste one readable image into each frame. Include the Remix transaction terminal and the expanded checker whenever needed to make the result clear.

Figure 2A: ETH HTLC compile and setup

Paste screenshot here

Show a successful compile with compiler version 0.8.20 or newer and the `SimpleHTLCChecker` address.

Figure 2B: ETH HTLC functional result

Paste screenshot here

Show the successful `runAllTests` transaction with 3 wei attached and `isSolved = true`.

Figure 2C: NFT HTLC and atomic-sale result

Paste screenshot here

Show the deployed `SimpleNFTHTLCChecker`, the successful `runAllTests` transaction with 1 wei attached, and `isSolved = true`.

Question 3: Smart-Contract Wallet and 2-of-3 Multisig

Background. An externally owned Ethereum account is controlled by one private key. A smart-contract wallet replaces that single point of control with code. In this question you progress from an owner-controlled ETH wallet, through ERC-20 token handling, to a wallet with three administrators. The final wallet requires approval from any two of the three administrators before it executes a payment. This reduces the risk that one lost or compromised key immediately loses all funds, while introducing the need to encode actions, count distinct approvals, and prevent replay.

Work to complete. Follow `lab/opt1/README.md` and complete the required choices and code in the three checkpoint folders under `lab/opt1/src/`. For the deployment demonstration, open `lab/opt1/src/2-multisig/MultisigWallet.sol` and its interface. Compile `MultisigWallet`, then deploy it with a JSON array containing three *different* Remix VM account addresses, for example `["0x...","0x...","0x..."]`.

Functional check. Send a small amount of test ETH to the deployed wallet. Encode a call to `transferEth(recipient, amount)` and use the same action bytes for every step. From the first administrator account, call `approve(action)`. Switch the Remix **Account** selector to the second administrator and call `execute(action)`; that call supplies the second approval and executes the payment. Verify that the recipient received the chosen amount and that calling `execute` again with the same action reverts. Your functional screenshot must show the two distinct administrator accounts or their approval events, the successful `Executed` event or transaction, and evidence of the resulting balance change. Also capture the failed replay attempt or the contract's `executed` value for the action hash.

Question 3 Screenshot Submission

Paste one readable image into each frame. A single Remix screenshot may contain several panels, but each frame must establish the evidence named below.

Figure 3A: Compile and deployment

Paste screenshot here

Show a successful compile, compiler version, the deployed `MultisigWallet` address, and the three distinct administrator addresses used for construction.

Figure 3B: Two approvals and execution

Paste screenshot here

Show calls or events from two different administrator accounts, the successful `Executed` transaction, and evidence that the recipient's balance changed.

Figure 3C: Replay prevention

Paste screenshot here

Show that repeating `execute(action)` reverts, or show that `executed` is `true` for the action hash after the successful payment.